SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyse syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I Perarasan V (192521404) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Perarasan V

Annexure A

EXPERIMENTS

Q1.You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

(i) Accept an arithmetic expression with operators +, -, *, / and parentheses

(ii) Use syntax-directed evaluation to compute values based on operator precedence

(iii) Optimize evaluation by reducing unnecessary temporary computations

(iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

E → E + T { E.val = E1.val + T.val }
E → E – T { E.val = E1.val – T.val }
E → T { E.val = T.val }

T → T * F { T.val = T1.val * F.val }
T → T / F { T.val = T1.val / F.val }
T → F { T.val = F.val }

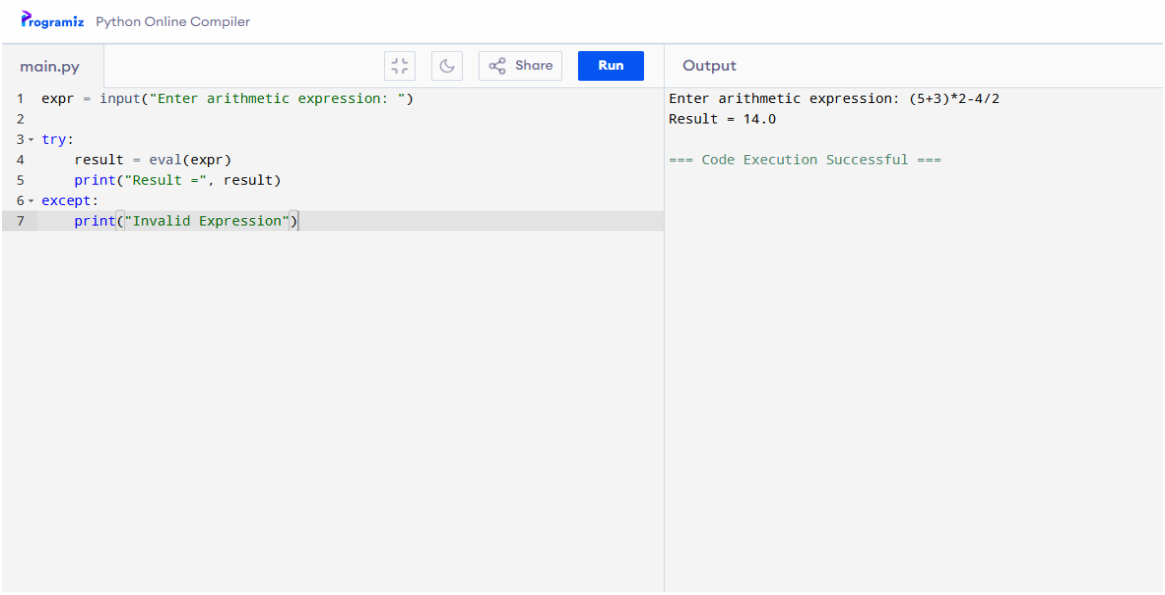
F → (E) { F.val = E.val }
F → digit { F.val = digit }

INPUT:

```
expr = input("Enter arithmetic expression: ")

try:
   result = eval(expr)
   print("Result =", result)
except:
   print("Invalid Expression")
```

OUTPUT:



Q4. In a compiler, it is important to check whether two type expressions are equivalent. Write a C program that:
(i).Accepts two type expressions (e.g., int, float, int*)
(ii).Compares them for type equivalence
(iii).Displays whether they are:

(a).Equivalent
(b).Not equivalent Extend the logic to handle pointer types (int*, float*).

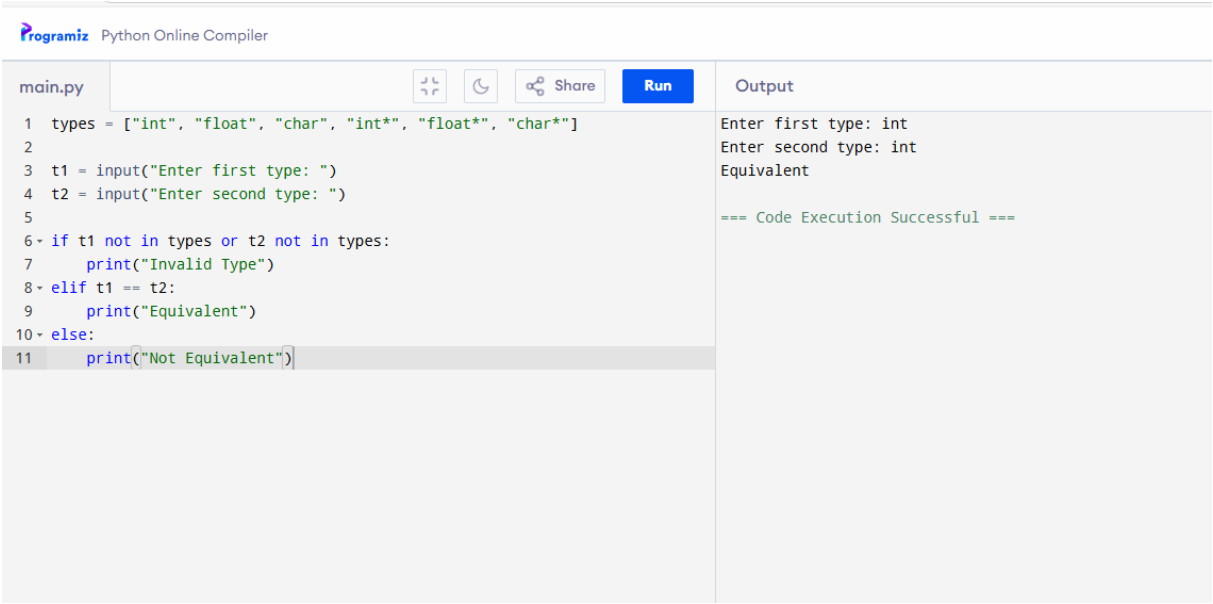
INPUT:

```
types = ["int", "float", "char", "int*", "float*", "char*"]

t1 = input("Enter first type: ")
t2 = input("Enter second type: ")

if t1 not in types or t2 not in types:
    print("Invalid Type")
elif t1 == t2:
    print("Equivalent")
else:
    print("Not Equivalent")
```

OUTPUT:



Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions. Write a C program that:
(i). Accepts two operands and their types
(ii). Checks whether the operation (e.g., +, *) is valid
(iii). Reports:

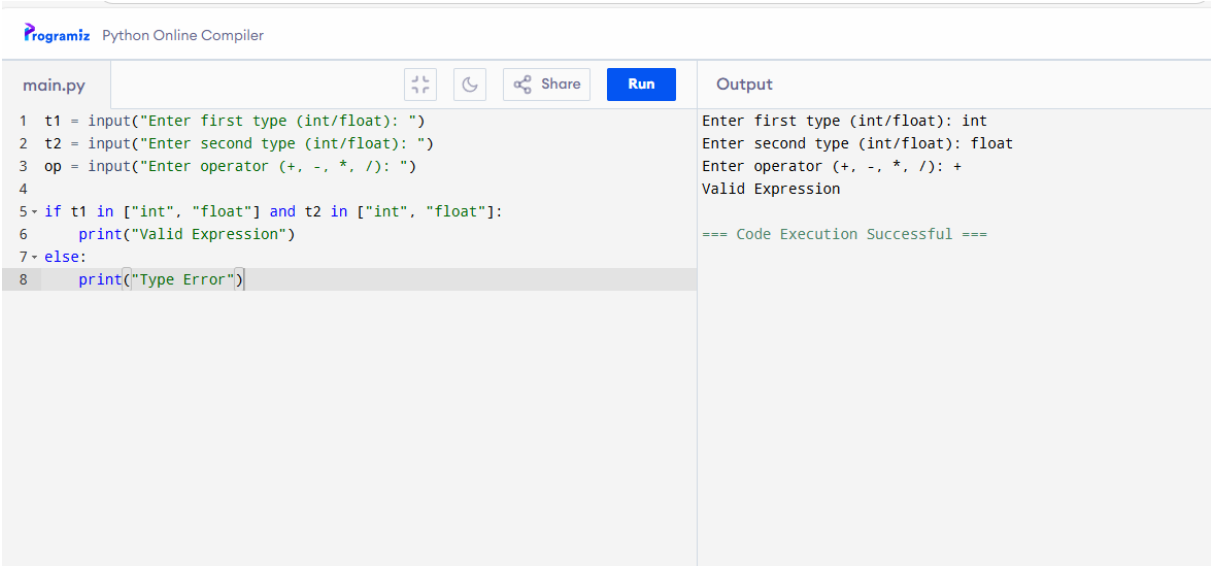
(a).Valid expression
(b).Type error (e.g., int + char*) Demonstrate how compilers detect semantic errors during type checking

INPUT:

```
t1 = input("Enter first type (int/float): ")
t2 = input("Enter second type (int/float): ")
op = input("Enter operator (+, -, *, /): ")

if t1 in ["int", "float"] and t2 in ["int", "float"]:
    print("Valid Expression")
else:
    print("Type Error")
```

OUTPUT:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation